

Migration de Base de Données

Étude de cas : Plateforme MedAssist

Migration d'un système de gestion de dossiers médicaux

Technologies : PostgreSQL 16 · Flyway 10 · Docker

Modalité : Travail en groupe de 3 étudiants

Livrables : Rapport de stratégie + Scripts SQL + Tests

Table des matières

1. Contexte et présentation de MedAssist
2. Architecture technique actuelle
3. Schéma de la base de données actuelle
4. Les évolutions demandées
5. Contraintes et exigences
6. Travail demandé
7. Livrables attendus
8. Critères d'évaluation
9. Annexe A – Environnement technique (Docker)
10. Annexe B – Rappel des stratégies de migration
11. Annexe C – Jeu de données initial

1. Contexte et présentation de MedAssist

MedAssist est une plateforme SaaS de gestion de dossiers médicaux utilisée par **350 cabinets médicaux** en France. Elle permet aux praticiens de gérer les dossiers patients, les consultations, les prescriptions et la facturation.

La plateforme est en production depuis 4 ans et traite en moyenne **12 000 consultations par jour**. L'application fonctionne selon une architecture classique avec une API REST (Spring Boot) connectée à une base PostgreSQL 16.

⚠ **Contexte critique** : MedAssist traite des données de santé soumises à la réglementation HDS (Hébergement de Données de Santé) et au RGPD. Toute perte de données ou incohérence est inacceptable. La disponibilité contractuelle (SLA) est de 99,9 %, soit un maximum de 8h44 d'indisponibilité par an.

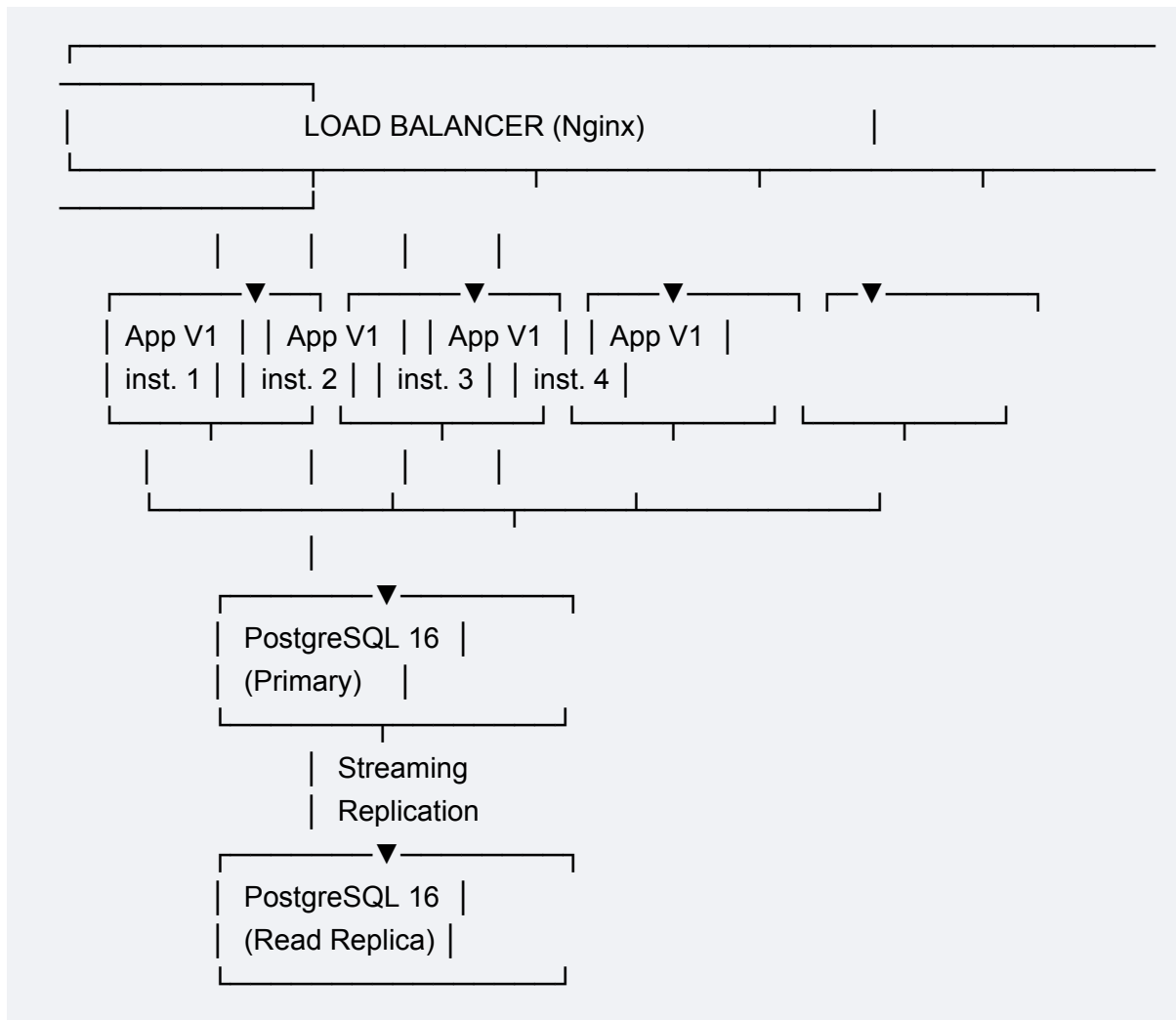
Chiffres clés

Métrique	Valeur
Nombre de cabinets clients	350
Patients en base	~2,4 millions
Consultations en base	~18 millions
Prescriptions en base	~45 millions
Consultations / jour	~12 000
Utilisateurs simultanés (pic)	~800
SLA contractuel	99,9 %
Fenêtre de maintenance autorisée	Dimanche 2h-6h (4 heures)
Instances applicatives	4 (derrière un load balancer)

2. Architecture technique actuelle

L'architecture de MedAssist suit un modèle classique multi-tiers :

Architecture actuelle



- **Déploiement** : Rolling update via Kubernetes – les 4 instances sont mises à jour une par une
- **Migrations BDD** : Flyway 10, exécuté au démarrage de la première instance déployée
- **Backup** : pg_dump quotidien + WAL archiving continu
- **Monitoring** : Prometheus + Grafana avec alertes sur les temps de réponse et les erreurs SQL

3. Schéma de la base de données actuelle

Voici le schéma simplifié des tables principales concernées par les évolutions demandées. D'autres tables existent (utilisateurs, facturation, etc.) mais ne sont pas impactées.

3.1 Table patients

```
CREATE TABLE patients (  
  id          BIGSERIAL  PRIMARY KEY,  
  first_name  VARCHAR(100) NOT NULL,  
  last_name   VARCHAR(100) NOT NULL,  
  birth_date  DATE       NOT NULL,  
  gender      CHAR(1)    NOT NULL,  -- 'M', 'F'  
  ssn         VARCHAR(15)  UNIQUE NOT NULL, -- Numéro de Sécurité Sociale  
  phone       VARCHAR(20),  
  email       VARCHAR(255),  
  address_line1 VARCHAR(255),  
  address_line2 VARCHAR(255),  
  city        VARCHAR(100),  
  postal_code  VARCHAR(10),  
  created_at   TIMESTAMP          NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
  updated_at   TIMESTAMP          NOT NULL DEFAULT  
CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_patients_name ON patients (last_name, first_name);  
CREATE INDEX idx_patients_ssn ON patients (ssn);
```

3.2 Table consultations

```

CREATE TABLE consultations (
  id          BIGSERIAL  PRIMARY KEY,
  patient_id  BIGINT     NOT NULL REFERENCES patients(id),
  doctor_name VARCHAR(200) NOT NULL,
  consultation_date TIMESTAMP NOT NULL,
  symptoms    TEXT,
  diagnosis    TEXT,
  notes        TEXT,
  consultation_type VARCHAR(50) NOT NULL, -- 'GENERAL', 'SPECIALIST',
  'EMERGENCY', 'FOLLOW_UP'
  fee_amount   DECIMAL(10,2) NOT NULL,
  fee_currency VARCHAR(3)     NOT NULL DEFAULT 'EUR',
  is_paid       BOOLEAN      NOT NULL DEFAULT FALSE,
  created_at    TIMESTAMP     NOT NULL DEFAULT
  CURRENT_TIMESTAMP
);

CREATE INDEX idx_consultations_patient ON consultations (patient_id);
CREATE INDEX idx_consultations_date ON consultations (consultation_date);
CREATE INDEX idx_consultations_doctor ON consultations (doctor_name);

```

3.3 Table prescriptions

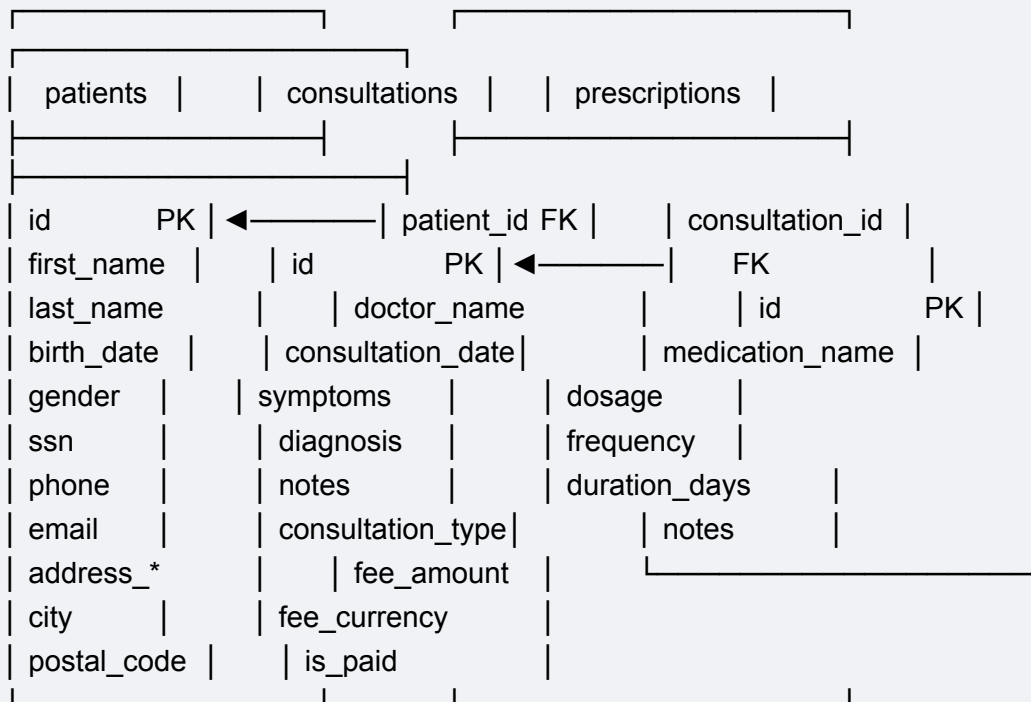
```

CREATE TABLE prescriptions (
  id          BIGSERIAL  PRIMARY KEY,
  consultation_id BIGINT     NOT NULL REFERENCES consultations(id),
  medication_name VARCHAR(255) NOT NULL,
  dosage       VARCHAR(100) NOT NULL,
  frequency     VARCHAR(100) NOT NULL, -- ex: '3 fois par jour'
  duration_days INTEGER     NOT NULL,
  notes         TEXT,
  created_at    TIMESTAMP     NOT NULL DEFAULT
  CURRENT_TIMESTAMP
);

CREATE INDEX idx_prescriptions_consultation ON prescriptions (consultation_id);

```

3.4 Diagramme des relations



4. Les évolutions demandées

L'équipe produit a validé les évolutions suivantes pour la version 2.0 de MedAssist. Ces changements doivent être implémentés sur la base de données existante, **avec les données en production**.

Évolution A – Restructuration de l'adresse des patients

Problème actuel : L'adresse est stockée dans des champs séparés directement dans la table patients (address_line1, address_line2, city, postal_code). Cela pose problème car :

- Un patient peut avoir plusieurs adresses (domicile, travail, adresse de facturation)
- L'application V2 doit permettre l'historisation des changements d'adresse
- Le format actuel ne gère pas les adresses internationales (pas de champ pays)

Cible V2 : Extraire les adresses dans une table dédiée addresses avec une relation 1-N vers patients. Chaque adresse aura un type (domicile, travail, facturation) et un indicateur is_primary.

Structure cible

```
-- Structure cible souhaitée pour la table addresses
CREATE TABLE addresses (
  id          BIGSERIAL  PRIMARY KEY,
  patient_id  BIGINT     NOT NULL REFERENCES patients(id),
  address_type VARCHAR(20) NOT NULL, -- 'HOME', 'WORK', 'BILLING'
  line1       VARCHAR(255) NOT NULL,
  line2       VARCHAR(255),
  city        VARCHAR(100) NOT NULL,
  postal_code  VARCHAR(10) NOT NULL,
  country     VARCHAR(100) NOT NULL DEFAULT 'France',
  is_primary   BOOLEAN   NOT NULL DEFAULT FALSE,
  created_at   TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP
);
```

Évolution B – Normalisation du champ doctor_name dans consultations

Problème actuel : Le champ doctor_name est un texte libre (VARCHAR). En production, on constate des incohérences :

Constat en production

```
-- Exemples de données incohérentes en production
SELECT DISTINCT doctor_name FROM consultations ORDER BY doctor_name;

-- Résultat :
-- 'Dr Martin'
-- 'Dr. Martin'
-- 'Dr MARTIN'
-- 'Dr Jean Martin'
-- 'Dr. Jean MARTIN'
-- 'Martin Jean'
-- ... (le même médecin apparaît sous 6 formes différentes)
```

Cible V2 : Créer une table doctors avec un identifiant unique, et remplacer doctor_name par une clé étrangère doctor_id. Les ~120 médecins existants doivent être correctement dédoublés et rattachés.

Structure cible

```
-- Structure cible souhaitée pour la table doctors
CREATE TABLE doctors (
  id          BIGSERIAL  PRIMARY KEY,
  rpps_number VARCHAR(11)    UNIQUE NOT NULL, -- N° RPPS (identifiant
national)
  first_name  VARCHAR(100) NOT NULL,
  last_name   VARCHAR(100) NOT NULL,
  specialty   VARCHAR(100),
  email       VARCHAR(255),
  created_at  TIMESTAMP      NOT NULL DEFAULT
CURRENT_TIMESTAMP
);
```

Évolution C – Refonte du champ gender dans patients

Problème actuel : Le champ gender est un CHAR(1) acceptant uniquement 'M' ou 'F'. Suite à une évolution réglementaire, l'application V2 doit supporter :

- 'M' (Masculin), 'F' (Féminin), 'NB' (Non-binaire), 'U' (Non renseigné / Inconnu)
- Le type CHAR(1) est trop restrictif pour les nouvelles valeurs
- La contrainte implicite sur les valeurs doit être formalisée via une table de référence

Cible V2 : Remplacer le CHAR(1) par un VARCHAR(10) ou une FK vers une table de référence gender_ref, et migrer les données existantes.

Évolution D – Ajout du chiffrement des données sensibles (SSN)

Problème actuel : Le numéro de Sécurité Sociale (ssn) est stocké en clair dans la table patients. Suite à un audit de sécurité, il est demandé de chiffrer cette donnée au repos (encryption at rest).

Cible V2 : Le champ ssn doit être chiffré via l'extension pgcrypto de PostgreSQL. L'application V2 gèrera le chiffrement/déchiffrement. Pendant la transition, les deux formats (clair et chiffré) doivent coexister.

Rappel pgcrypto


```
-- Exemple d'utilisation de pgcrypto pour le chiffrement
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Chiffrement
SELECT pgp_sym_encrypt('1234567890123', 'ma_cle_secrete');

-- Déchiffrement
SELECT pgp_sym_decrypt(colonne_chiffree::bytea, 'ma_cle_secrete');
```

Évolution E – Partitionnement de la table consultations

Problème actuel : La table consultations contient 18 millions de lignes et grossit de ~12 000 lignes/jour. Les requêtes de reporting (agrégations par mois/année) deviennent très lentes. L'équipe infrastructure demande un partitionnement par année.

Cible V2 : Transformer consultations en table partitionnée par RANGE sur consultation_date, avec une partition par année (2021, 2022, 2023, 2024, 2025, et une partition future). Les données existantes doivent être redistribuées dans les partitions.

⚠ Attention : PostgreSQL ne permet pas de convertir une table classique en table partitionnée via un simple ALTER TABLE. Il faut créer une nouvelle structure et migrer les données.

5. Contraintes et exigences

5.1 Contraintes non négociables

#	Contrainte	Justification
C1	Zéro perte de données	Réglementation HDS – données de santé
C2	Disponibilité maximale pendant la migration	SLA 99,9 % – les cabinets ne peuvent pas se permettre d'arrêt
C3	Rollback possible à chaque étape	Obligation de pouvoir revenir en arrière en cas de problème
C4	Compatibilité V1/V2 pendant la transition	Rolling deploy : V1 et V2 coexistent pendant le déploiement
C5	Intégrité référentielle garantie	Pas de données orphelines, pas de FK cassées
C6	Performance : aucune dégradation > 10 %	Temps de réponse moyen actuel : 120ms (API)

5.2 Contraintes techniques

- **PostgreSQL 16** : Seules les fonctionnalités natives de PG16 sont disponibles (pas d'extension tierce sauf pgcrypto)
- **Flyway 10** : Toutes les migrations doivent être versionnées et reproductibles via Flyway
- **Docker** : L'environnement de développement est fourni (voir Annexe A)
- **Pas d'outil ETL externe** : La migration doit être réalisable uniquement avec des scripts SQL et Flyway

5.3 Fenêtre de maintenance

Une fenêtre de maintenance est disponible chaque **dimanche de 2h à 6h** (4 heures). Pendant cette fenêtre, le service peut être dégradé ou brièvement interrompu. En dehors de cette fenêtre, le service doit rester pleinement opérationnel.

 **Remarque** : Certaines évolutions peuvent nécessiter plusieurs fenêtres de maintenance successives. Planifiez votre stratégie en conséquence.

6. Travail demandé

Votre équipe est missionnée pour **concevoir, implémenter et tester la stratégie de migration** de la base de données MedAssist de la V1 vers la V2.

Partie 1 – Analyse et choix de stratégie (rapport écrit)

Pour chacune des 5 évolutions (A à E), vous devez :

- 1. Analyser les risques** – Identifier les risques spécifiques à cette évolution (verrouillage, perte de données, incompatibilité, performance...).
- 2. Choisir une stratégie de migration** – Parmi les stratégies connues, choisir celle qui convient le mieux et justifier votre choix.
- 3. Justifier pourquoi les autres stratégies sont moins adaptées** – Montrer que vous avez considéré les alternatives.
- 4. Planifier le séquençement** – Définir l'ordre des étapes et identifier les dépendances entre évolutions.

5. Estimer l'impact – Évaluer si la migration peut se faire en ligne (sans interruption) ou si elle nécessite la fenêtre de maintenance.

⚡ **Important** : Il n'y a pas une seule bonne réponse. Plusieurs stratégies peuvent être valides pour une même évolution. Ce qui compte, c'est la qualité de votre argumentation et la cohérence de votre raisonnement.

Partie 2 – Implémentation technique (scripts SQL)

Implémenter les scripts de migration Flyway pour **au moins 3 évolutions sur les 5** (au choix). Pour chaque évolution implémentée :

1. Scripts Flyway versionnés – Respecter la convention de nommage Flyway (V1__, V2__, etc.). Chaque script doit pouvoir être exécuté de manière idempotente.

2. Scripts de migration de données – Backfill, transformation, nettoyage des données existantes.

3. Mécanismes de compatibilité – Triggers de synchronisation, vues de compatibilité, ou tout autre mécanisme nécessaire pour assurer la coexistence V1/V2.

4. Scripts de rollback – Pour chaque étape, fournir le script permettant de revenir à l'état précédent.

Partie 3 – Validation et tests

Pour chaque évolution implémentée, fournir un script de test SQL qui vérifie :

- L'intégrité des données après migration (aucune perte, aucune corruption)
- La compatibilité ascendante (les requêtes V1 fonctionnent encore pendant la transition)
- La compatibilité descendante (les requêtes V2 fonctionnent correctement)
- Le bon fonctionnement du rollback (retour à l'état antérieur sans perte)
- Les performances (les requêtes clés ne sont pas dégradées de plus de 10 %)

Format des tests : Utiliser des requêtes SQL avec des assertions sous forme de DO \$\$... \$\$ qui lèvent une EXCEPTION en cas d'échec, ou des SELECT retournant TRUE/FALSE.

7. Livrables attendus et modalité de rendu

#	Livrable	Format	Description
L1	Rapport de stratégie	PDF ou Word	Analyse, choix et justification pour les 5 évolutions (A-E). Inclut le plan de séquençement global.
L2	Scripts Flyway	Fichiers .sql	Scripts versionnés pour au moins 3 évolutions. Placés dans flyway/sql/.
L3	Scripts de rollback	Fichiers .sql	Un script de rollback par étape de migration.
L4	Scripts de tests	Fichiers .sql	Tests de validation pour chaque étape migrée.
L5	docker-compose.yml	YAML	Environnement de travail fonctionnel (peut être celui fourni en annexe, modifié si besoin).

Nom du repo : MedAssist-nom-nom-nom

Rendu par mail : rida@lamerkanterie.fr

Organisation des fichiers

```
projet-medassist/
├── docker-compose.yml
├── rapport/
│   └── rapport_strategie.pdf
├── flyway/
│   ├── conf/
│   │   └── flyway.conf
│   └── sql/
│       ├── V1__init_schema.sql           # Schéma initial (fourni)
│       ├── V1.1__seed_data.sql          # Données de test (fourni)
│       ├── V2__evolution_A_expand.sql    # Exemple de nommage
│       ├── V3__evolution_A_backfill.sql
│       ├── V4__evolution_A_contract.sql
│       ├── V5__evolution_B_step1.sql
│       ├── ...
│       └── rollback/
│           ├── R_V2__rollback.sql
│           ├── R_V3__rollback.sql
│           └── ...
└── tests/
```

```
|— test_evolution_A.sql
|— test_evolution_B.sql
|— ...
```

8. Critères d'évaluation

Critère	Points	Détails
Analyse des risques	/3	Identification correcte et complète des risques pour chaque évolution
Choix de stratégie	/4	Pertinence du choix, qualité de la justification, prise en compte des alternatives
Séquencement global	/2	Cohérence de l'ordre des migrations, gestion des dépendances entre évolutions
Scripts Flyway	/4	Qualité du SQL, respect des conventions Flyway, idempotence, commentaires
Migration de données	/3	Exactitude du backfill, gestion des cas limites et des données incohérentes
Mécanismes de compatibilité	/2	Triggers, vues, ou autres mécanismes de coexistence V1/V2
Scripts de rollback	/2	Complétude et fonctionnalité du retour arrière
Tests de validation	/3	Couverture, pertinence et automatisation des tests
Qualité du rapport	/2	Clarté, structure, schémas, professionnalisme
TOTAL	/25	

💡 **Bonus** : Implémentation des 5 évolutions au lieu de 3, ou mise en place de tests de performance comparatifs (EXPLAIN ANALYZE avant/après migration).

9. Annexe A – Environnement technique (Docker)

L'environnement de développement suivant est fourni. Il contient PostgreSQL 16, Flyway 10, et l'extension pgcrypto préinstallée.

docker-compose.yml

version: '3.9'

services:

postgres:

image: postgres:16

container_name: medassist_pg

environment:

POSTGRES_USER: medassist_user

POSTGRES_PASSWORD: medassist_pass

POSTGRES_DB: medassist

ports:

- "5434:5432"

volumes:

- medassist_data:/var/lib/postgresql/data

command: >

postgres

-c shared_preload_libraries='pg_stat_statements'

-c pg_stat_statements.track=all

flyway:

image: flyway/flyway:10

container_name: medassist_flyway

depends_on:

- postgres

volumes:

- ./flyway/sql:/flyway/sql

- ./flyway/conf:/flyway/conf

command: [

"-configFiles=/flyway/conf/flyway.conf",

"-connectRetries=10",

"migrate"

]

volumes:

medassist_data:

flyway/conf/flyway.conf

```
flyway.url=jdbc:postgresql://postgres:5432/medassist  
flyway.user=medassist_user  
flyway.password=medassist_pass  
flyway.locations=filesystem:/flyway/sql  
flyway.schemas=public  
flyway.baselineOnMigrate=true
```

Commandes utiles

```
# Démarrer l'environnement  
docker-compose up -d postgres  
docker-compose run --rm flyway migrate  
  
# Se connecter à PostgreSQL  
docker exec -it medassist_pg psql -U medassist_user -d medassist  
  
# Voir l'état des migrations Flyway  
docker-compose run --rm flyway info  
  
# Exécuter une migration spécifique (target)  
docker-compose run --rm flyway -target=3 migrate  
  
# Relancer Flyway après ajout de scripts  
docker-compose run --rm flyway migrate
```

10. Annexe B – Rappel des stratégies de migration

Cette annexe présente un rappel synthétique des principales stratégies de migration de schéma. Elle est fournie à titre de référence ; vous êtes libres d'utiliser d'autres approches si elles sont pertinentes et justifiées.

Expand-Contract (Parallel Change)

Description : Ajouter les nouvelles structures (expand), migrer les données et les applications (transition), puis supprimer les anciennes structures (contract).

✓ Avantages	Zero-downtime, rollback facile, compatible rolling deploy
✗ Inconvénients	Plus complexe, nécessite des triggers de synchronisation, schéma temporairement "gonflé"
🎯 Cas d'usage typique	Changements de colonnes/types avec coexistence V1/V2 nécessaire

Blue-Green Database

Description : Maintenir deux copies de la base (blue = production, green = nouvelle version). Basculer le trafic d'un coup après validation.

✓ Avantages	Rollback instantané (rebascule vers blue), test complet avant bascule
✗ Inconvénients	Coûteux en ressources (double infra), synchronisation des écritures complexe
🎯 Cas d'usage typique	Changements majeurs nécessitant un test complet avant mise en production

Big Bang (avec fenêtre de maintenance)

Description : Arrêter le service, exécuter toute la migration d'un coup, relancer le service.

✓ Avantages	Simple à implémenter, pas de coexistence à gérer
✗ Inconvénients	Downtime requis, pas de rollback facile, risqué sur gros volumes
🎯 Cas d'usage typique	Petites tables, changements simples, ou quand le downtime est acceptable

Shadow Table + Rename/Swap

Description : Créer une nouvelle table avec la structure cible (shadow), copier/migrer les données, puis renommer (swap) les tables.

✓ Avantages	Permet de restructurer en profondeur (partitionnement, etc.), pas de modification de la table source
✗ Inconvénients	Nécessite un mécanisme pour capturer les écritures pendant la copie, bref downtime au moment du swap
🎯 Cas d'usage typique	Changements de structure majeurs (partitionnement, changement de PK, etc.)

Vues de compatibilité

Description : Créer des vues SQL qui simulent l'ancien schéma sur la nouvelle structure (ou inversement).

✓ Avantages	Transparent pour l'application, pas de duplication de données
✗ Inconvénients	Limité aux SELECT (INSERT/UPDATE complexes à gérer via INSTEAD OF triggers), performances variables
🎯 Cas d'usage typique	Quand l'ancienne application ne peut pas être modifiée rapidement

Migration progressive par lots (Backfill batching)

Description : Migrer les données par petits paquets (batches) plutôt qu'en une seule transaction massive.

✓ Avantages	Réduit le verrouillage, permet de migrer en arrière-plan, applicable sur très gros volumes
✗ Inconvénients	Plus lent au total, nécessite de gérer la cohérence pendant la migration partielle
🎯 Cas d'usage typique	Tables de millions/milliards de lignes où un UPDATE massif est impraticable

11. Annexe C – Jeu de données initial

Le script suivant (V1.1__seed_data.sql) insère un jeu de données représentatif pour tester vos migrations. Les données réelles de production contiennent ~2,4M patients, ~18M consultations et ~45M prescriptions, mais ce jeu réduit suffit pour valider la logique de migration.

V1__init_schema.sql

```

--
=====
=====
-- V1__init_schema.sql : Création du schéma initial
--
=====
=====

CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE patients (
    id          BIGSERIAL  PRIMARY KEY,
    first_name  VARCHAR(100) NOT NULL,
    last_name   VARCHAR(100) NOT NULL,
    birth_date  DATE       NOT NULL,
    gender      CHAR(1)    NOT NULL CHECK (gender IN ('M', 'F')),
    ssn         VARCHAR(15)  UNIQUE NOT NULL,
    phone       VARCHAR(20),
    email       VARCHAR(255),
    address_line1 VARCHAR(255),
    address_line2 VARCHAR(255),
    city        VARCHAR(100),
    postal_code  VARCHAR(10),
    created_at   TIMESTAMP          NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    updated_at   TIMESTAMP          NOT NULL DEFAULT
CURRENT_TIMESTAMP
);

CREATE INDEX idx_patients_name ON patients (last_name, first_name);
CREATE INDEX idx_patients_ssn ON patients (ssn);

CREATE TABLE consultations (
    id          BIGSERIAL  PRIMARY KEY,
    patient_id   BIGINT    NOT NULL REFERENCES patients(id),
    doctor_name  VARCHAR(200) NOT NULL,
    consultation_date TIMESTAMP NOT NULL,
    symptoms     TEXT,
    diagnosis     TEXT,
    notes        TEXT,
    consultation_type VARCHAR(50) NOT NULL,
    fee_amount    DECIMAL(10,2) NOT NULL,
    fee_currency  VARCHAR(3)  NOT NULL DEFAULT 'EUR',

```

```

        is_paid          BOOLEAN    NOT NULL DEFAULT FALSE,
        created_at       TIMESTAMP          NOT NULL DEFAULT
CURRENT_TIMESTAMP
    );

CREATE INDEX idx_consultations_patient ON consultations (patient_id);
CREATE INDEX idx_consultations_date ON consultations (consultation_date);
CREATE INDEX idx_consultations_doctor ON consultations (doctor_name);

CREATE TABLE prescriptions (
    id          BIGSERIAL          PRIMARY KEY,
    consultation_id BIGINT          NOT NULL REFERENCES consultations(id),
    medication_name VARCHAR(255) NOT NULL,
    dosage      VARCHAR(100)       NOT NULL,
    frequency   VARCHAR(100)       NOT NULL,
    duration_days INTEGER          NOT NULL,
    notes       TEXT,
    created_at  TIMESTAMP          NOT NULL DEFAULT
CURRENT_TIMESTAMP
);

CREATE INDEX idx_prescriptions_consultation ON prescriptions (consultation_id);

```

V1.1__seed_data.sql

```

--
=====
=====
-- V1.1__seed_data.sql : Jeu de données de test
--
=====
=====

-- —— Patients

```

```

INSERT INTO patients (first_name, last_name, birth_date, gender, ssn, phone, email,
                      address_line1, address_line2, city, postal_code) VALUES
('Marie', 'Dupont', '1985-03-15', 'F', '285037512345', '0601020304',
 'marie.dupont@email.com', '12 Rue de la Paix', NULL, 'Paris', '75002'),
('Jean', 'Martin', '1972-08-22', 'M', '172086912345', '0611223344',
 'jean.martin@email.com', '5 Avenue des Champs', 'Apt 3B', 'Lyon', '69001'),
('Sophie', 'Bernard', '1990-11-30', 'F', '290117512345', '0622334455',
 'sophie.b@email.com', '28 Rue du Commerce', NULL, 'Marseille', '13001'),
('Pierre', 'Leroy', '1968-05-10', 'M', '168056912345', '0633445566',
 'p.leroy@email.com', '3 Place de la République', NULL, 'Toulouse', '31000'),
('Isabelle', 'Moreau', '1995-01-25', 'F', '295017512345', '0644556677',
 'isa.moreau@email.com', '15 Boulevard Victor Hugo', 'Bât C', 'Nantes', '44000'),
('François', 'Garcia', '1960-12-03', 'M', '160126912345', NULL,
 NULL, '7 Impasse des Lilas', NULL, 'Bordeaux', '33000'),
('Camille', 'Roux', '2001-07-18', 'F', '201077512345', '0666778899',
 'camille.roux@email.com', NULL, NULL, NULL, NULL),
('Antoine', 'Fournier', '1988-09-08', 'M', '188096912345', '0677889900',
 'a.fournier@email.com', '42 Rue Pasteur', NULL, 'Lille', '59000');

-- —— Consultations (avec incohérences volontaires sur doctor_name) ——
INSERT INTO consultations (patient_id, doctor_name, consultation_date, symptoms,
                          diagnosis, notes, consultation_type, fee_amount, is_paid) VALUES
(1, 'Dr Martin', '2024-01-15 09:00', 'Maux de tête fréquents',
 'Migraines chroniques', 'Prescription antalgiques', 'GENERAL', 25.00, TRUE),
(1, 'Dr. Martin', '2024-03-20 14:30', 'Suivi migraines',
 'Amélioration', 'Continuer traitement', 'FOLLOW_UP', 25.00, TRUE),
(2, 'Dr MARTIN', '2024-02-10 10:00', 'Douleur thoracique',
 'Intercostal', 'RAS', 'EMERGENCY', 50.00, FALSE),
(2, 'Dr Jean Martin', '2024-06-01 11:00', 'Contrôle annuel',
 'Bonne santé', NULL, 'GENERAL', 25.00, TRUE),
(3, 'Dr. Dubois', '2024-01-22 15:00', 'Allergie cutanée',

```

```

'Dermatite', 'Crème corticoïde', 'SPECIALIST', 50.00, TRUE),
(3, 'Dr Dubois', '2024-04-10 09:30', 'Suivi dermatite',
'Résolution', NULL, 'FOLLOW_UP', 25.00, TRUE),
(4, 'Dubois Claire', '2024-03-05 08:00', 'Entorse cheville',
'Entorse grade 2', 'Attelle + repos', 'EMERGENCY', 50.00, FALSE),
(5, 'Dr. Jean MARTIN', '2024-05-15 16:00', 'Fatigue chronique',
'Carence fer', 'Bilan sanguin', 'GENERAL', 25.00, TRUE),
(5, 'Dr Martin', '2024-07-20 10:00', 'Suivi carence',
'Normalisation', 'Arrêt supplémentation', 'FOLLOW_UP', 25.00, TRUE),
(6, 'Dr Petit Anne', '2024-02-28 14:00', 'Douleurs articulaires',
'Arthrose débutante', 'Anti-inflammatoires', 'SPECIALIST', 50.00, TRUE),
(7, 'Dr. Dubois', '2024-08-01 09:00', 'Vaccination',
NULL, 'Rappel DTP', 'GENERAL', 25.00, FALSE),
(8, 'Martin Jean', '2024-04-22 11:30', 'Lombalgie',
'Lombalgie aiguë', 'Kiné prescrite', 'GENERAL', 25.00, TRUE),
(1, 'Dr. Martin', '2023-06-15 09:00', 'Contrôle annuel',
'RAS', NULL, 'GENERAL', 25.00, TRUE),
(2, 'Dr Martin', '2023-11-10 14:00', 'Grippe',
'Grippe saisonnière', 'Repos + paracétamol', 'GENERAL', 25.00, TRUE),
(4, 'Dr Petit Anne', '2023-09-20 10:00', 'Bilan santé',
'Cholestérol limite', 'Régime alimentaire', 'GENERAL', 25.00, TRUE);

```

-- — Prescriptions

```

INSERT INTO prescriptions (consultation_id, medication_name, dosage,
frequency, duration_days, notes) VALUES
(1, 'Paracétamol', '1000mg', '3 fois par jour', 10, 'Pendant les crises'),
(1, 'Sumatriptan', '50mg', 'Au besoin', 30, 'Max 2/jour'),
(3, 'Ibuprofène', '400mg', '2 fois par jour', 5, NULL),
(5, 'Bétaméthasone crème', '0.05%', '2 applications/jour', 14, 'Zone affectée'),
(7, 'Kétoprofène gel', '2.5%', '3 applications/jour', 10, 'Cheville gauche'),
(8, 'Fer Fumarate', '66mg', '1 fois par jour', 90, 'À jeun'),
(10, 'Diclofénac', '50mg', '2 fois par jour', 14, 'Pendant les repas'),
(12, 'Paracétamol', '1000mg', '3 fois par jour', 5, NULL),
(12, 'Thiocolchicoside', '4mg', '2 fois par jour', 7, NULL);

```

 **Note :** Observez attentivement les incohérences dans la colonne doctor_name des consultations. Elles sont volontaires et représentent l'état réel des données à migrer. Votre stratégie pour l'évolution B devra traiter ce problème de dédoublement.

